A Coefficient-Aware Finite-Difference Benchmark for Solver Selection and CPU/GPU Stencil Scaling in Heat-Conduction Simulation

Dong Liu*
University of Nottingham Ningbo China
Ningbo, China
ssydl3@nottingham.edu.cn

Abstract

Sparse solves for variable-coefficient heat conduction are sensitive to coefficient structure, grid resolution, preconditioner setup cost, and implementation path. This paper presents a verified and reproducible finite-difference benchmark that connects these factors under controlled forcing and bounded hardware measurements. Smooth manufactured solutions recover second-order L^2 convergence, while an aligned two-material test verifies discontinuous-interface flux treatment and motivates an arithmetic-versus-harmonic face-averaging sensitivity check. Across the tested coefficient families, contrast alone does not determine conditioning, and the preferred setup-inclusive solver changes with resolution: Jacobi-PCG is selected throughout the smallest single-pass decision grid, whereas the tested PyAMG smoothed-aggregation PCG configuration dominates $N = 128$ and $N = 256$. Representative repeated timings support the difficult-case selections, while the averaging sensitivity preserves the solver-class transition. CUDA results are reported only for resident-data stencil kernels and are not interpreted as full sparse-solver timings. The contribution is a controlled benchmark methodology rather than a new discretisation, Krylov method, AMG algorithm, or GPU kernel. The scripts, raw CSV files, tests, generated tables, and figures are provided as a supplementary artefact.

1 Introduction

Heat transfer, diffusion, and potential-flow approximations remain central modelling components in electrical, mechanical, and computer-aided engineering. Even when the governing physics is written compactly, the numerical workload becomes a sequence of large sparse linear systems or repeated stencil updates. For short-cycle engineering studies, the scientific-computing question is therefore not only whether a discretisation converges. It is also whether the solver, preconditioner, and implementation path scale predictably under reproducible conditions.

The research question is: under what coefficient and resolution conditions does a stronger preconditioner become worthwhile once setup cost is included, and are those solver-selection conclusions robust to interface averaging and implementation boundaries?

This paper evaluates a compact, self-contained benchmark for the steady heat-conduction equation with spatially varying conductivity. The model is simple enough to allow manufactured-solution verification and still exposes solver behaviour under smooth, inclusion, layered, and checkerboard coefficient fields. The implementation uses Python, NumPy, SciPy, PyAMG, Numba, and Numba-CUDA, reflecting a toolchain that is accessible on a normal workstation while still supporting sparse solvers and hardware-specific kernels [1, 2, 3, 4]. Recent large-scale examples motivate the same combined view: AMR-Wind and ICON-on-GPU report performance-portable engineering solvers, while GPU finite-element, tokamak-fluid, and immersed-boundary studies expose accelerator-specific PDE solver constraints [5, 6, 7, 8, 9].

Prior numerical-analysis texts establish the convergence properties of finite-difference discretisations and the role of Krylov and multigrid methods in sparse PDE solves [10, 11, 12, 13, 14]. Recent sparse and PDE solver studies further underline that preconditioning, sparse matrix execution, and accelerator mapping remain active constraints for modern workloads [15, 16]; recent AMG and diffusion-solver work

targets communication cost and coefficient difficulty more directly [17, 18]. However, many small engineering simulation papers report only a final plot or a single runtime. That practice makes it difficult to separate discretisation error, iterative convergence, preconditioner effect, and implementation throughput. General reproducible-benchmarking work provides principles for automated reruns and evidence retention [19]; the narrower contribution here is to instantiate that discipline inside a PDE heat-conduction benchmark whose claims link directly to discretisation, solver selection, coefficient descriptors, and local hardware measurements. The gap considered here is deliberately bounded: one heat-conduction problem family is used to connect PDE verification, solver scaling, and CPU/GPU stencil performance.

We do not propose a new finite-difference scheme, Krylov method, AMG algorithm, or GPU kernel. The paper instead makes five scoped benchmark contributions. First, it implements a conservative finite-difference operator for $-\nabla \cdot (k\nabla u)$, verifies second-order convergence with smooth manufactured solutions, and adds an analytic jump-interface check for face-averaging behaviour. Second, it introduces a coefficient-family grid and difficulty descriptors for smooth, inclusion, layered, and checkerboard conductivity fields, with an arithmetic/harmonic sensitivity check for the discontinuous high-contrast cases. Third, it connects those descriptors to solver behaviour through pooled and fixed-grid descriptive rank correlations with fixed-grid case-resampling intervals; the contribution is the stratified diagnostic protocol, not a claim that one scalar descriptor is sufficient on its own. Fourth, it evaluates a setup-inclusive single-solve decision protocol for CG, Jacobi-preconditioned CG, and the tested PyAMG-preconditioned CG configuration, with repeated timing checks for representative cases. Fifth, it measures CPU and CUDA stencil throughput under explicit conditions and fits a local crossover interpolation. The scope is a workstation-scale structured-grid case; no industrial geometry handling, unstructured finite elements, or production-solver robustness is claimed.

2 Related Work

Finite-difference methods provide a direct route from elliptic and parabolic PDEs to sparse linear systems on structured grids [10]. For self-adjoint diffusion operators, conservative flux forms are especially useful because face-based coefficient averaging preserves the symmetry and locality expected by Krylov methods. Once discretised, the main computational problem is sparse linear algebra rather than pointwise equation evaluation.

The conjugate-gradient method remains a baseline solver for symmetric positive definite systems [11, 12]. Its performance depends strongly on spectral conditioning, which generally deteriorates under mesh refinement and can be strongly affected by coefficient contrast and geometry. Multigrid methods reduce this sensitivity by addressing error components across scales [13]. Algebraic multigrid extends that idea to matrix-defined problems without requiring a hand-built geometric hierarchy [14]. In this study, PyAMG is used as an accessible Python implementation of algebraic multigrid [4].

Performance portability is a separate concern. NumPy provides high-level array operations, SciPy provides sparse linear algebra, and Numba compiles numerical Python kernels to machine code [2, 1, 3]. CUDA exposes massively parallel GPU execution, but launch overhead and data movement can erase speedups on small problems [20, 21]. Recent stencil and CUDA kernel studies report sensitivity to memory traffic, kernel structure, and problem size [22, 23]. The benchmark therefore reports GPU results as kernel-only measurements for repeated updates where data stay on the device, and it keeps raw evidence for independent reruns [19].

3 Mathematical Model and Discretisation

The model problem is the steady heat-conduction equation

$$-\nabla \cdot (k(x,y)\nabla u(x,y)) = f(x,y), \quad (x,y) \in (0,1)^2, \tag{1}$$

with homogeneous Dirichlet boundary conditions unless otherwise stated. The coefficient $k(x,y)$ represents spatially varying thermal conductivity. The smooth test uses

$$k(x,y) = 1 + 0.5\sin(2\pi x)\sin(2\pi y), \tag{2}$$

and the high-contrast stress test uses a circular inclusion,

$$k(x,y) = 1 + 99\,\mathbf{1}_{(x-0.5)^2+(y-0.5)^2<0.15^2}. \tag{3}$$

For the decision-map study, this pair is expanded into a parameterised coefficient grid. The smooth family uses $k = 1 + a\sin(2\pi x)\sin(2\pi y)$ with $a = (C_k - 1)/(C_k + 1)$, giving target contrast C_k. The inclusion, layered, and checkerboard families use piecewise values in $\{1, C_k\}$ with $C_k \in \{10, 30, 100\}$, while the smooth family uses $C_k \in \{3, 10, 30\}$ and the constant case gives $C_k = 1$.

To report contrast, coefficient sharpness, and coefficient geometry separately, each coefficient field is summarised by

$$C_k = \frac{\max k}{\min k}, \qquad G_k^{(h)} = \|\nabla_h \log k\|_\infty, \qquad V_k = \sum |\Delta_x k| + \sum |\Delta_y k|, \tag{4}$$

where $G_k^{(h)}$ is a discrete log-gradient sharpness indicator and V_k is used only as a discrete total-variation proxy. For discontinuous coefficient fields, $G_k^{(h)}$ depends on the grid spacing and is interpreted at fixed N rather than as a continuous invariant. Small-grid spectral evidence is also recorded through the estimated condition number $\kappa(A) = \lambda_{\max}(A)/\lambda_{\min}(A)$.

The domain is discretised on an $N \times N$ uniform grid with spacing $h = 1/(N-1)$. Interior unknowns are ordered lexicographically. For an interior node (i, j), the conservative five-point stencil is

$$\begin{aligned}(Au)_{i,j} = {} & k_{i+1/2,j}\frac{u_{i,j} - u_{i+1,j}}{h^2} + k_{i-1/2,j}\frac{u_{i,j} - u_{i-1,j}}{h^2} \\ & + k_{i,j+1/2}\frac{u_{i,j} - u_{i,j+1}}{h^2} + k_{i,j-1/2}\frac{u_{i,j} - u_{i,j-1}}{h^2},\end{aligned} \tag{5}$$

where face coefficients use arithmetic averages of adjacent nodal values in the main two-dimensional decision map. Arithmetic averaging is retained as the primary benchmark convention to preserve continuity with the smooth-coefficient stencil and the original coefficient-family decision grid. For discontinuous coefficient fields, harmonic face averaging is also implemented and tested as a sensitivity check, because it matches the series-resistance interpretation of one-dimensional layered conduction. Both choices produce sparse symmetric positive definite systems for positive k and homogeneous Dirichlet boundaries.

For smooth convergence verification, the exact solution is

$$u_{\text{exact}}(x,y) = \sin(\pi x)\sin(\pi y), \tag{6}$$

and f is generated analytically for the constant and smooth coefficient cases. Errors are reported in discrete L^2 and maximum norms over interior nodes. The observed convergence rate is obtained by a least-squares fit of $\log(e)$ against $\log(h)$. A separate one-dimensional verification problem uses $k(x) = k_1$ for $x < 1/2$ and $k(x) = k_2$ for $x > 1/2$ with $u(0) = 0$ and $u(1) = 1$. Its analytic solution is piecewise linear and satisfies continuity of both u and ku_x at the interface; this test isolates whether arithmetic and harmonic face averages respect the expected interface flux.

4 Solver and Implementation Methods

The assembled sparse matrix is solved with three methods: unpreconditioned CG, Jacobi-preconditioned CG, and algebraic-multigrid preconditioned CG. The AMG case uses the tested PyAMG default smoothed-aggregation hierarchy as a V-cycle preconditioner for SciPy CG; this should be read as one concrete AMG configuration rather than a claim about all AMG parameter choices. Manufactured-solution runs use a relative residual tolerance of 10^{-11} so that solver error does not dominate discretisation error; solver-comparison runs use SciPy CG with relative tolerance 10^{-8}, zero absolute tolerance, the default zero initial guess, and a maximum of 12000 iterations in the decision map. All coefficient-family solver comparisons use the same deterministic unit right-hand side at a given grid size, so changes in iteration count are not confounded by case-specific forcing. The reported time separates preconditioner setup from iterative solve time and also gives the total time. Failed convergence would be recorded as a structured row in the raw CSV artefact, although all reported benchmark cases converged.

Matrix-free performance is measured with a separate stencil apply matching Eq. (5). Three CPU paths are compared: NumPy vectorisation, Numba serial compilation, and Numba parallel compilation with controlled thread counts. Throughput is reported as an estimated stencil bandwidth using a fixed byte model: 11 double-precision values per interior point for the variable-coefficient stencil, or 88 bytes per point. The Jacobi crossover uses five double-precision values per interior point, or 40 bytes per point. These estimates are operational metrics for comparing implementations of the same stencil family, not hardware peak-bandwidth or direct DRAM-transaction claims.

The CUDA experiment uses a repeated Jacobi update rather than the full variable-coefficient operator. This isolates a simple memory-bound stencil and avoids conflating kernel throughput with sparse-solver algorithmics. CPU and GPU arrays are allocated once, warm-up iterations are run, and timing covers repeated kernel execution. Host-device transfers are excluded from the speedup calculation. This boundary matches repeated time-stepping or smoother-like updates where data remain resident on the device, but it should not be read as end-to-end application speedup.

The setup-inclusive single-solve decision protocol is deliberately simple. For coefficient case c, grid size N, and method set $\mathcal{S}$, it selects

$$s^*(c, N) = \arg\min_{s \in \mathcal{S}} \left[T_{\text{setup}}(s, c, N) + T_{\text{solve}}(s, c, N) \right], \tag{7}$$

among converged runs. This makes one-time AMG setup cost visible rather than treating iteration count as the only outcome. It is not a repeated-right-hand-side reuse model; such a model would require an explicit solve count m.

To make the coefficient descriptors operational rather than decorative, the benchmark also reports Spearman rank correlations between C_k, $G_k^{(h)}$, V_k, the available-grid condition estimate, CG iterations, and the speedup of the selected solver over CG. The coefficient set is a designed benchmark grid rather than a random sample from a defined population, so rank correlations are treated as descriptive association diagnostics rather than population-level inferential estimates. Pooled rows summarise all case-size entries but are not treated as independent samples because the same coefficient family appears at multiple grid sizes. Fixed-N rows provide the safer coefficient-level reading and include deterministic case-resampling intervals. Iteration-count associations are treated as the primary solver-difficulty diagnostic; associations with selected-solver speedup are secondary and exploratory because the full decision grid uses a single timing pass per cell. Permutation p-values are archived in the CSV as diagnostics rather than used to claim pairwise descriptor separation. Timing stability is assessed by repeating selected decision-map cases five times and then reporting the solver chosen by median total time, the per-repeat vote fraction, and the relative interquartile range of the selected method. The full decision grid remains a single-pass benchmark, while the repeat check probes whether the most important decisions are robust or near ties. The hardware crossover model is similarly bounded: for the repeated Jacobi kernel it fits $T(n) = \alpha + \beta n$, where $n = (N-2)^2$, and uses the fitted curves only to interpolate the measured CPU/CUDA equal-time region.

5 Experiments and Results

All experiments were run on Windows 11 with Python 3.12.9, NumPy 2.4.6, SciPy 1.18.1, Numba 0.67.0, PyAMG 5.3.0, and Numba-CUDA 0.30.4. The machine used an AMD Ryzen 9 7940HS w/ Radeon 780M Graphics CPU with 8 physical cores and 16 logical processors, approximately 16 GB of system memory reported by the operating system as 15.2 GB usable, and an NVIDIA GeForce RTX 4060 Laptop GPU with compute capability 8.9. The exact raw CSV outputs, environment metadata, and scripts are included in the supplementary artefact. The decision map covers 13 coefficient cases, three grid sizes, and three Krylov/preconditioner choices. The spectral conditioning study uses $N = 32, 64, 128$. The rank-correlation analysis uses all 39 case-size decisions and three fixed-grid strata of 13 cases; condition numbers are matched to $N = 64$ and $N = 128$ where available, and the $N = 128$ condition estimate is used as the maximum-available case-level spectral proxy for $N = 256$. The repeated timing check uses five repeats for one constant case and four high-difficulty cases. The face-averaging sensitivity study reruns the three discontinuous $C_k = 100$ stress cases with arithmetic and harmonic face averages at $N = 64, 128, 256$.

Table 1: Manufactured-solution convergence for the finite-difference discretisation.

Case	p_{L^2}	p_∞	Finest L^2	Residual
Constant coefficient	2.01	2.00	6.35×10^{-6}	1.60×10^{-12}
Smooth variable coefficient	2.02	2.00	6.39×10^{-6}	2.01×10^{-11}

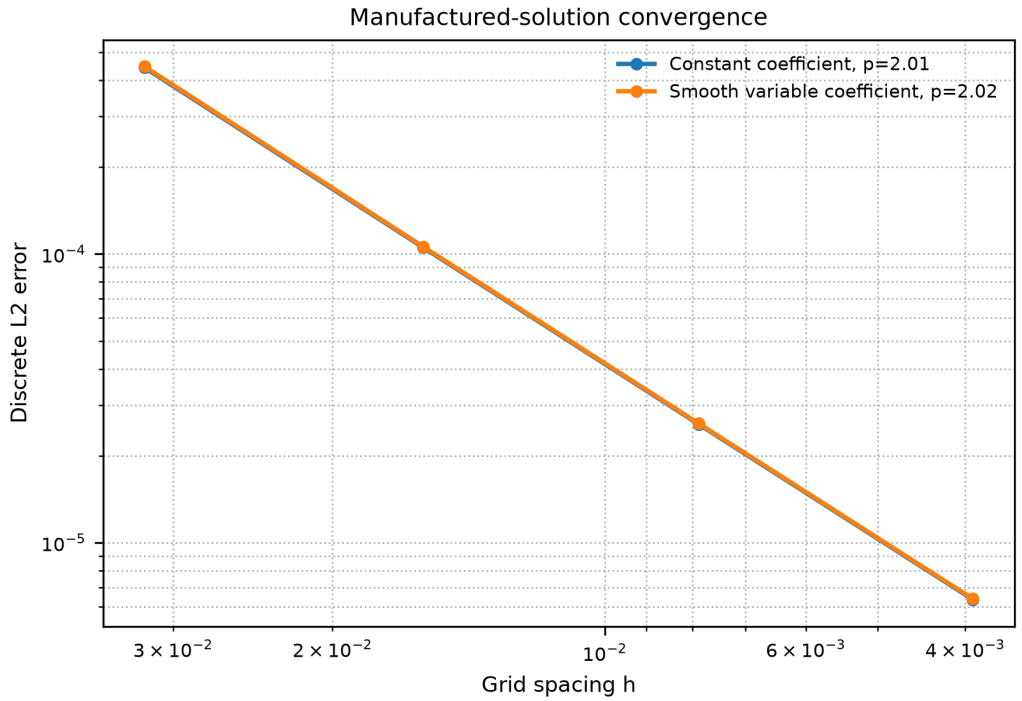

Figure 1: Manufactured-solution convergence of the finite-difference operator.

5.1 Verification and Coefficient Difficulty

The manufactured-solution study confirms the expected second-order behaviour of the finite-difference scheme. Table 1 reports fitted rates using $N = 32, 64, 128, 256$ with a CG residual tolerance of 10^{-11}. Both the constant and smooth variable-coefficient cases produced L^2 rates close to two. Figure 1 gives the same result as a log-log error plot.

The discontinuous-interface check in Table 2 isolates a different error mechanism. For an aligned one-dimensional two-material conduction problem, harmonic face averaging reproduced the analytic interface flux to roundoff, while arithmetic averaging showed a grid-refined interface error. This does not make harmonic averaging a universal choice for all multidimensional discontinuities, but it prevents the benchmark from relying only on smooth manufactured-solution evidence.

Table 3 reports representative coefficient descriptors. The spectral evidence confirms that contrast alone is not a complete difficulty measure. At $N = 128$, the inclusion case with $C_k = 100$ had an estimated condition number of 6.43×10^5, while the checkerboard case with the same contrast had a much smaller estimate of 3.12×10^4 under this discretisation. Figure 2 shows the same effect across the measured coefficient grid.

Table 4 turns the coefficient descriptors into a solver-behaviour diagnostic and also shows why a single pooled ranking is unsafe. Across all 39 case-size entries, the matched condition estimate had the largest association with selected-solver speedup ($\rho = 0.94$), while $G_k^{(h)}$ had a larger pooled association with CG iterations ($\rho = 0.82$) than contrast ($\rho = 0.63$). Within fixed-N strata, the condition estimate, contrast,

Table 2: Discontinuous-interface verification on a one-dimensional two-material heat-conduction problem. Harmonic face averaging recovers the analytic flux continuity to roundoff in this aligned-interface test.

N	L^2 error, arithmetic	L^2 error, harmonic	Flux error, arithmetic	Flux error, harmonic
32	1.27×10^{-2}	5.96×10^{-16}	6.33×10^{-2}	5.11×10^{-15}
64	6.21×10^{-3}	1.56×10^{-15}	3.07×10^{-2}	6.66×10^{-15}
128	3.07×10^{-3}	3.55×10^{-14}	1.51×10^{-2}	2.44×10^{-14}
256	1.53×10^{-3}	9.51×10^{-15}	7.49×10^{-3}	4.91×10^{-14}

Table 3: Representative coefficient-difficulty descriptors. Metrics use $N = 64$ for the discrete descriptors; $\kappa(A)$ is the largest available spectral estimate, here $N = 128$.

Case	Family	C_k	$G_k^{(h)}$	TV proxy	$\kappa(A)$
constant	Constant	1.0	0.00	0.00	6.54×10^3
smooth_c30	Smooth	29.7	16.41	4.76	1.38×10^4
inclusion_c100	Inclusion	100.0	205.15	113.14	6.43×10^5
layered_c100	Layered	100.0	145.06	100.57	2.61×10^5
checkerboard_c100	Checkerboard	100.0	205.15	1408.00	3.12×10^4

and $G_k^{(h)}$ were all positively associated with CG iterations, with median stratum correlations of $\rho = 0.96$, 0.89, and 0.83, respectively. However, the fixed-grid samples contain only 13 coefficient cases and the case-resampling intervals overlap in the raw CSV archive. The result is therefore used as a descriptive ordering signal and as evidence for stratified reporting, not as a formal claim that these descriptors are significantly separated.

5.2 Solver Decision Map and Preconditioner Effect

The single-pass setup-inclusive decision map gives a sharper result than a single solver comparison. Over the 13 coefficient cases, Jacobi-PCG was fastest in every $N = 64$ run, while AMG-PCG was fastest in every $N = 128$ and $N = 256$ run. Table 5 summarises this transition. The median speedup of the selected solver over CG increased from 2.79× at $N = 64$ to 7.80× at $N = 256$, and the largest speedup reached 25.72× for the layered $C_k = 100$ case. Because each decision-map cell uses one timing pass, this map is interpreted together with the repeated representative cases below.

Table 6 tests whether the discontinuous high-contrast solver conclusions are an artefact of arithmetic face averaging. Replacing arithmetic by harmonic averaging changed condition estimates and iteration counts, especially for checkerboard fields, but the fastest solver class was unchanged in all nine tested case-size cells: Jacobi-PCG remained fastest at $N = 64$, and AMG-PCG remained fastest at $N = 128$ and $N = 256$. The stronger claim is not that averaging is irrelevant, but that the reported solver-decision transition survived this targeted interface-discretisation perturbation.

The repeat timing check in Table 7 gives a stricter interpretation of the map. For the high-difficulty representative cases at $N = 256$, AMG-PCG remained the median-best solver in every repeat, with selected-method relative IQR between 0.5% and 6.6%. The constant-coefficient $N = 64$ case was a near tie: Jacobi-PCG was best by median time, but the per-repeat vote favoured CG in four of five repeats and the median speedup was only 1.03×. The benchmark therefore treats that small-grid constant case as a timing boundary rather than a robust preconditioner preference.

Solver behaviour changed substantially when the coefficient field became high contrast. At $N = 256$, unpreconditioned CG required 4370 iterations in the high-contrast case, compared with 770 iterations in the smooth case. Jacobi preconditioning reduced the high-contrast iteration count to 535, while AMG-PCG reduced it to 12. The timing reduction was also large: total time fell from 6.155 s for CG to 0.288 s for AMG-PCG in the high-contrast case. Table 8 summarises the finest-grid comparison, and Fig. 4 and

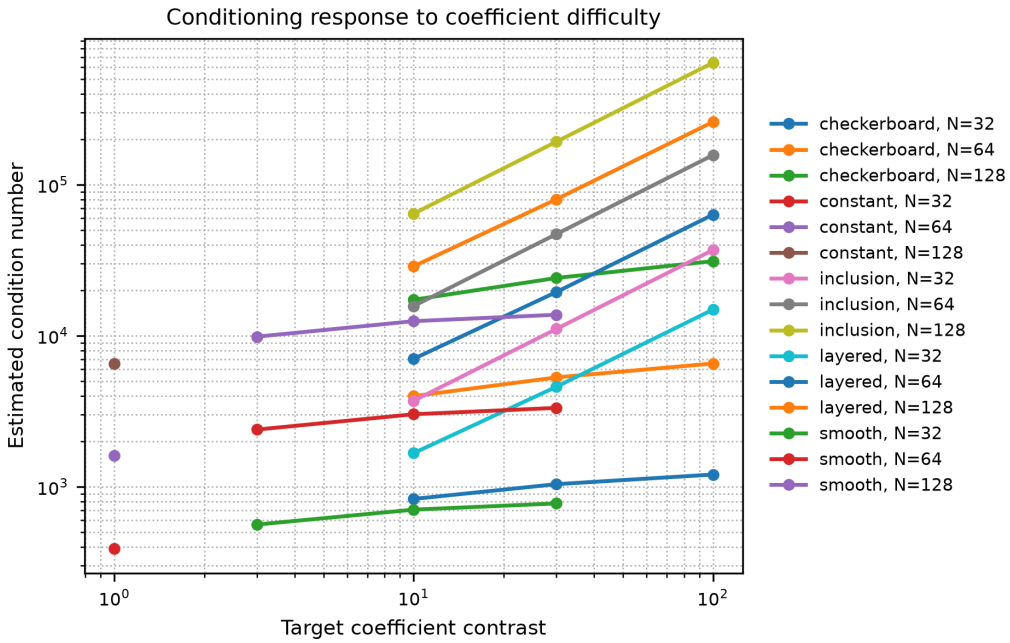

Figure 2: Estimated condition number response to coefficient contrast and coefficient geometry.

Table 4: Descriptive Spearman rank correlations linking coefficient-difficulty descriptors to solver behaviour. Pooled columns use all 39 case-size entries and are not treated as independent population samples; fixed-N columns summarise the stratum-specific rank coefficients by median and, when they differ, range. Raw stratum values and case-resampling uncertainty intervals are archived in the supplementary CSV.

Descriptor	CG pooled	CG fixed-N	Speedup pooled	Speedup fixed-N
C_k contrast	0.63	0.89	0.68	0.85 (0.82-0.86)
$G_k^{(h)}$ log-gradient	0.82	0.83	0.80	0.77 (0.74-0.77)
V_k TV proxy	0.43	0.65	0.42	0.53 (0.47-0.57)
$\kappa(A)$ proxy	0.87	0.96 (0.95-0.96)	0.94	0.95 (0.94-0.96)

Fig. 5 show scaling across grid sizes.

5.3 CPU and CUDA Stencil Scaling

The matrix-free CPU stencil benchmark shows that implementation path can dominate the cost of repeated PDE stencil operations. At $N = 2048$, NumPy required 0.1813 s per apply, while Numba serial required 0.005325 s and Numba parallel with four threads required 0.003725 s. Increasing thread count beyond four did not improve the largest case in this run, which is consistent with bandwidth saturation and thread-management overhead rather than a numerical failure.

The CUDA Jacobi benchmark passed the predefined inclusion rule. At $N = 512$, GPU and CPU kernel times were essentially equal. At $N = 1024$, CUDA reached 1.52× speedup. At $N = 2048$ and $N = 4096$, the speedups increased to 3.23× and 3.51×, respectively. These results justify including GPU scaling as a bounded kernel-level result. They do not imply the same speedup for an end-to-end sparse solver.

The local linear crossover fit gives a compact way to report this boundary. Table 10 gives the fitted slopes and equal-time estimate. The fitted crossover is an illustrative interpolation near $N = 755$; the directly measured evidence is that $N = 512$ is near equal time and $N = 1024$ is already faster on CUDA.

Table 5: Setup-inclusive single-solve decision summary over the coefficient-family grid. Counts report the fastest converged method by setup-plus-solve time from one timing pass per cell.

N	Cases	CG	Jacobi-PCG	AMG-PCG	Median speedup vs CG	Max speedup vs CG
64	13	0	13	0	2.79	6.76
128	13	0	0	13	5.44	19.40
256	13	0	0	13	7.80	25.72

Table 6: Arithmetic/harmonic face-averaging sensitivity for discontinuous $C_k = 100$ stress cases. Each paired entry reports arithmetic/harmonic values. The full CSV also records $\kappa(A)$ using matched N for 64 and 128 and the $N = 128$ proxy for $N = 256$.

Case	N	CG it.	Jacobi-PCG it.	AMG-PCG it.	Best solver	Best speedup
inclusion_c100	64	796/725	129/129	10/10	Jacobi-PCG/Jacobi-PCG	6.02/3.92
inclusion_c100	128	1963/1891	264/263	12/11	AMG-PCG/AMG-PCG	14.25/14.45
inclusion_c100	256	4370/4326	535/535	12/12	AMG-PCG/AMG-PCG	18.40/20.74
layered_c100	64	1136/1120	164/181	9/9	Jacobi-PCG/Jacobi-PCG	6.61/6.43
layered_c100	128	2447/2442	325/367	10/10	AMG-PCG/AMG-PCG	17.21/18.58
layered_c100	256	5055/5054	657/715	10/10	AMG-PCG/AMG-PCG	24.44/23.67
checkerboard_c100	64	443/557	171/190	13/28	Jacobi-PCG/Jacobi-PCG	2.20/2.78
checkerboard_c100	128	1050/1308	350/388	17/32	AMG-PCG/AMG-PCG	6.83/6.03
checkerboard_c100	256	2268/2809	718/783	17/31	AMG-PCG/AMG-PCG	8.44/7.81

This fit is used as an interpolation summary over the measured range, not as a claim about hardware launch overhead outside the experiment.

6 Discussion

The main numerical result is the decision-level transition rather than the superiority of a single method in isolation. At small grids, Jacobi-PCG can win because its setup cost is negligible, but the repeated timing check shows that this statement has a near-tie boundary for the constant $N = 64$ case. Once the grid reaches the tested $N = 128$ and $N = 256$ levels, AMG-PCG wins consistently in the decision grid and in the repeated representative checks because the solve-time reduction outweighs the one-time AMG setup cost. This behaviour is consistent with the multilevel purpose of AMG: reduce error across scales rather than relying on local diagonal rescaling.

The coefficient-difficulty results also show why neither contrast nor any other single descriptor should be used alone. Inclusion and checkerboard fields can share both nominal contrast and discrete log-gradient values while having very different condition estimates, because geometry changes the operator spectrum. The pooled rank correlations are useful for summarising the combined grid-size and coefficient-field effect, but the fixed-grid correlations give the safer coefficient-level reading: contrast, discrete sharpness, and spectral conditioning capture different parts of the solver response, and the small sample size prevents a strong significance claim about their pairwise ordering. This is the main methodological lesson of the descriptor analysis: benchmark descriptors should be reported with stratification and uncertainty, not collapsed into a single universal difficulty score.

The performance results add a second layer. Compiled stencil kernels are much faster than NumPy for repeated local updates, but parallel CPU scaling is not monotone. On the largest CPU stencil case, four threads outperformed eight and sixteen threads. The safest interpretation is stencil-bandwidth and scheduling saturation under this memory-bound kernel. The CUDA result is also size-dependent. It becomes useful only after the grid is large enough to offset launch overhead, and the reported speedup assumes data residency on the GPU. The fitted crossover is therefore a reporting device for this specific kernel and workstation, not a general GPU superiority claim.

The benchmark has clear boundaries. It uses a two-dimensional structured grid, synthetic coefficient fields, and workstation-scale experiments. Harmonic averaging is not a substitute for an interface-fitted

Table 7: Repeated timing stability check. Rows report the solver selected by median total time across repeats, the per-repeat vote fraction, and the relative interquartile range of the selected method.

Case	N	Median best	Vote best	Vote	Speedup	Rel. IQR	Status
constant	64	Jacobi-PCG	CG	0.80	1.03	33.5%	variable
smooth_c30	256	AMG-PCG	AMG-PCG	1.00	5.03	2.2%	stable
inclusion_c100	256	AMG-PCG	AMG-PCG	1.00	19.85	0.5%	stable
layered_c100	256	AMG-PCG	AMG-PCG	1.00	24.05	1.2%	stable
checkerboard_c100	256	AMG-PCG	AMG-PCG	1.00	8.98	6.6%	stable

Table 8: Solver comparison at $N = 256$ grid points in each direction.

Case	Method	Iter.	Time (s)	Rel. residual
Smooth variable	CG	770	0.939	9.71×10^{-9}
Smooth variable	Jacobi-PCG	605	0.829	9.88×10^{-9}
Smooth variable	AMG-PCG	9	0.240	4.82×10^{-9}
High contrast	CG	4370	6.155	9.95×10^{-9}
High contrast	Jacobi-PCG	535	0.728	9.14×10^{-9}
High contrast	AMG-PCG	12	0.288	4.44×10^{-9}

or immersed-interface discretisation when curved material boundaries cut the Cartesian grid. The benchmark does not claim industrial geometry handling, unstructured finite elements, transient multiphysics coupling, or production solver robustness. Its value is instead methodological: it gives a reproducible bridge from PDE discretisation to solver choice and hardware execution, with each claim tied to a raw CSV table, a script, and, for the descriptor analysis, explicit uncertainty checks.

7 Reproducibility Artefact

The supplementary artefact contains the scripts, raw CSV files, generated tables and figures, metadata, and tests needed to reproduce the reported benchmark. The main scripts cover convergence, solver comparison, the coefficient-aware decision map, conditioning, interface verification, descriptor analysis, arithmetic/harmonic sensitivity, timing stability, CPU/CUDA stencil scaling, crossover fitting, and table/figure regeneration. Unit tests cover the numerical operators, solver behaviour, summaries, plotting, tables, descriptors, conditioning, timing stability, and crossover fit; every reported manuscript number traces to a raw CSV file.

8 Conclusion

This paper presented a coefficient-aware finite-difference benchmark for heat-conduction simulation. Smooth verification, an aligned-interface check, coefficient descriptors, setup-inclusive solver decisions, repeated timings, and bounded CPU/CUDA stencil measurements were linked to raw CSV evidence. The main lesson is methodological: solver choice should be reported with discretisation checks, coefficient stratification, setup cost, timing stability, and explicit hardware boundaries. In this benchmark, Jacobi-PCG was selected on the smallest single-pass grid, AMG-PCG dominated the tested larger grids, and CUDA speedups were meaningful only for resident-data stencil kernels at sufficiently large problem sizes.

Table 9: Representative stencil-throughput measurements. CPU entries use the variable-coefficient operator at $N = 2048$; CUDA entries use a repeated Jacobi kernel-only benchmark.

Experiment	Method	N	Time	Throughput / speedup
CPU stencil	NumPy	2048	0.1813 s/apply	2.03 GB/s
CPU stencil	Numba serial	2048	0.005325 s/apply	69.18 GB/s
CPU stencil	Numba parallel, 4 threads	2048	0.003725 s/apply	98.90 GB/s
GPU crossover	CUDA kernel	2048	7.97×10^{-4} s/step	3.23× vs CPU
GPU crossover	CUDA kernel	4096	3.15×10^{-3} s/step	3.51× vs CPU

Table 10: Empirical linear CPU/CUDA kernel crossover model fitted over the measured Jacobi-stencil range. The fit is used as a local interpolation of the crossover point, not as a physical launch-overhead model.

Component	Observations	β (s/unknown)	R^2	Crossover N
CPU Numba	4	6.780×10^{-10}	0.999	-
CUDA kernel	4	1.896×10^{-10}	0.999	-
Illustrative fitted crossover	-	-	0.999	755

References

[1] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020.

[2] Charles R. Harris, K. Jarrod Millman, Stefan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[3] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pages 1–6, 2015.

[4] Nathan Bell, Luke N. Olson, Jacob Schroder, and Ben Southworth. PyAMG: Algebraic multigrid solvers in python. *Journal of Open Source Software*, 8(87):5495, 2023.

[5] Michael B. Kuhn, Marc T. Henry de Frahan, Prakash Mohan, Georgios Deskos, Matt Churchfield, Lawrence Cheung, Ashesh Sharma, Ann Almgren, Shreyas Ananthan, Michael J. Brazell, et al. AMR-Wind: A performance-portable, high-fidelity flow solver for wind farm simulations. *Wind Energy*, 28(5), 2025.

[6] Xavier Lapillonne, Daniel Hupp, Fabian Gessler, Andre Walser, Andreas Pauling, Annika Lauber, Benjamin Cumming, Carlos Osuna, Christoph Muller, Claire Merker, et al. Operational numerical weather prediction with ICON on GPUs (version 2024.10). *Geoscientific Model Development*, 19(2):755–772, 2026.

[7] Leisheng Li, Ximeng Fu, Xiyu Zheng, Huiyuan Li, and Jinxi Li. GPU-accelerated finite-element method for the three-dimensional unstructured mesh atmospheric dynamic framework. *Geoscientific Model Development*, 19(15):7525–7544, 2026.

[8] Mark F. Adams, Jin Chen, and Benjamin Sturdevant. Fast solvers for tokamak fluid models with PETSc. *Computer Physics Communications*, 327:110293, 2026.

[9] Sushrut Kumar, Joshua Romero, Jung-Hee Seo, Massimiliano Fatica, and Rajat Mittal. A GPU-accelerated sharp interface immersed boundary solver for large scale flow simulations. In *AIAA SCITECH 2026 Forum*. American Institute of Aeronautics and Astronautics, 2026.

[10] John C. Strikwerda. *Finite Difference Schemes and Partial Differential Equations*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 edition, 2004.

[11] Magnus R. Hestenes and Eduard Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952.

[12] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 edition, 2003.

[13] William L. Briggs, Van Emden Henson, and Steve F. McCormick. *A Multigrid Tutorial*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 edition, 2000.

[14] John W. Ruge and Klaus Stuben. Algebraic multigrid. In Stephen F. McCormick, editor, *Multigrid Methods*, volume 3 of *Frontiers in Applied Mathematics*, pages 73–130. Society for Industrial and Applied Mathematics, Philadelphia, PA, 1987.

[15] Massimo Bernaschi, Alessandro Celestini, Giorgio Richelli, and Pasqua D'Ambra. On the energy efficiency of sparse matrix computations on multi-GPU clusters. *Future Generation Computer Systems*, 183:108519, 2026.

[16] Andrew Welter and Ngoc Cuong Nguyen. Preconditioning techniques for hybridizable discontinuous Galerkin discretizations on GPU architectures. *Computer Methods in Applied Mechanics and Engineering*, 456:118951, 2026.

[17] Fan Yuan, Xiaojian Yang, Yunqing Huang, Dezun Dong, Chuanfu Xu, Jie Liu, Xiaoqiang Yue, Shengguo Li, and Hongxia Wang. CRAMG: A communication-reduced algebraic multigrid method. In *Proceedings of the 39th ACM International Conference on Supercomputing*, pages 397–411. ACM, 2025.

[18] David Green, Xiaozhe Hu, Jeremy Lore, Lin Mu, and Mark L. Stowell. An efficient high-order numerical solver for diffusion equations with strong anisotropy. *Computer Physics Communications*, 276:108333, 2022.

[19] Tuomas Koskela, Ilektra Christidi, Mose Giordano, Emily Dubrovska, Jamie Quinn, Christopher Maynard, Dave Case, Kaan Olgu, and Tom Deakin. Principles for automated and reproducible benchmarking. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, pages 609–618. ACM, 2023.

[20] NVIDIA Corporation. *CUDA C++ Programming Guide*, 2026. Accessed 3 September 2026.

[21] NVIDIA Corporation. *Numba-CUDA Documentation*, 2026. Accessed 3 September 2026.

[22] Johannes Pekkila, Oskar Lappi, Fredrik Robertsen, and Maarit J. Korpi-Lagg. Stencil computations on AMD and Nvidia graphics processors: Performance and tuning strategies. *Concurrency and Computation: Practice and Experience*, 37(12–14), 2025.

[23] Yerlan Makhmut, Timur Imankulov, Sergei Gorlatch, and Bazargul Matkerim. A CUDA performance study of global- and shared-memory kernels for the Buckley-Leverett polymer-flooding problem. *Applied Sciences*, 16(11):5449, 2026.

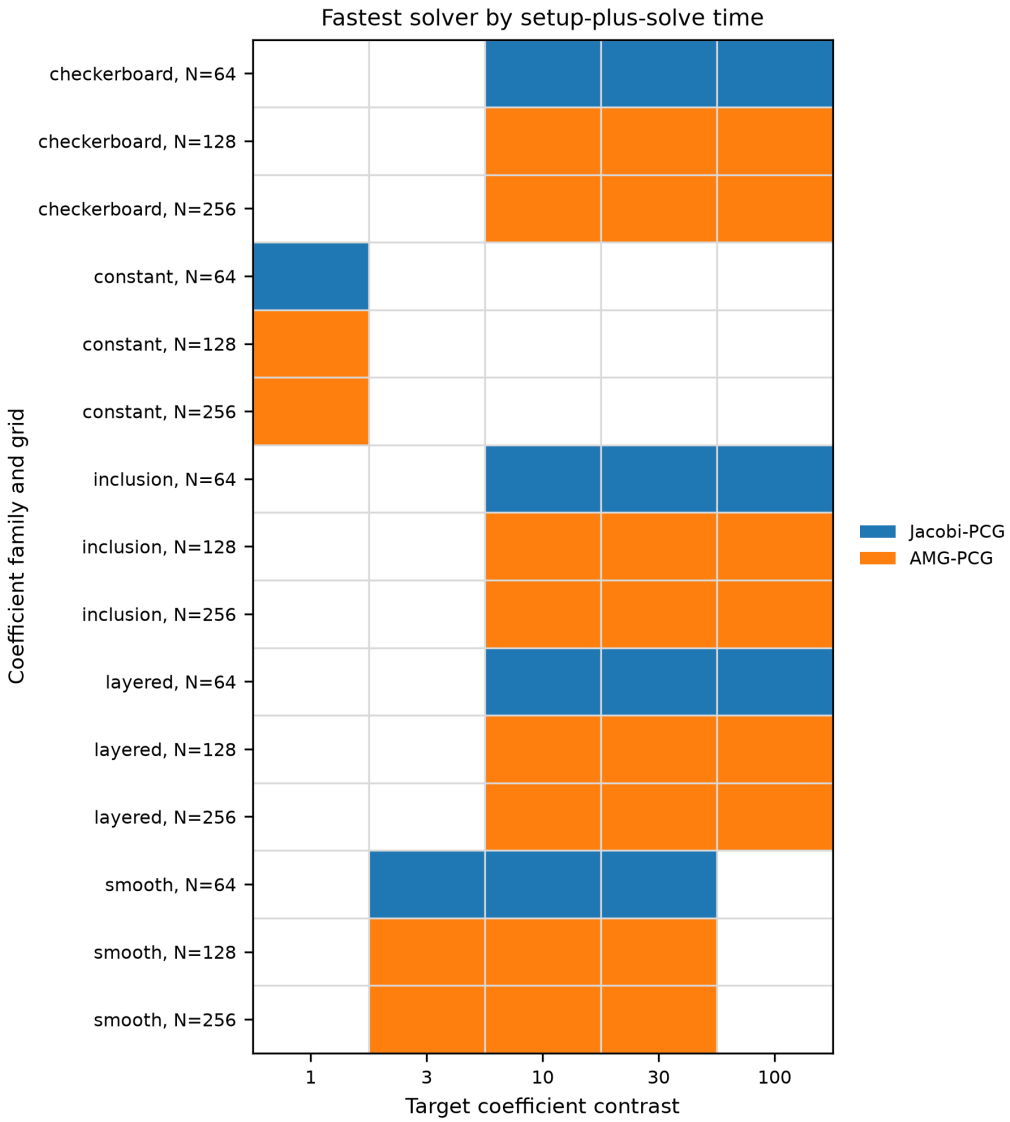

Figure 3: Fastest converged solver by setup-plus-solve time over the coefficient-family grid. Blank cells correspond to contrast levels not used for that family.

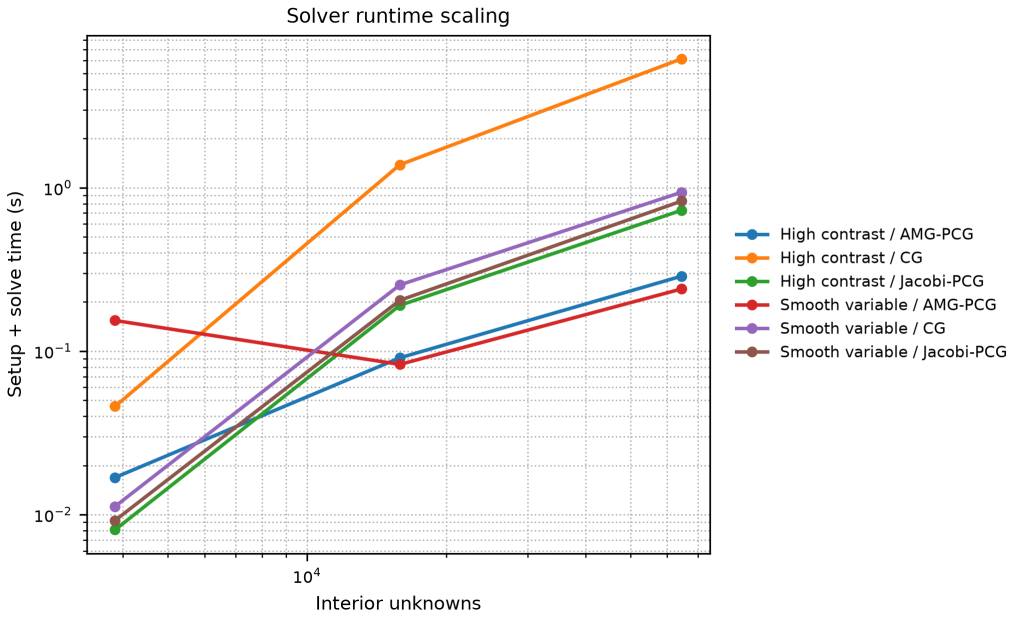

Figure 4: Total setup plus solve time for CG, Jacobi-PCG, and AMG-PCG.

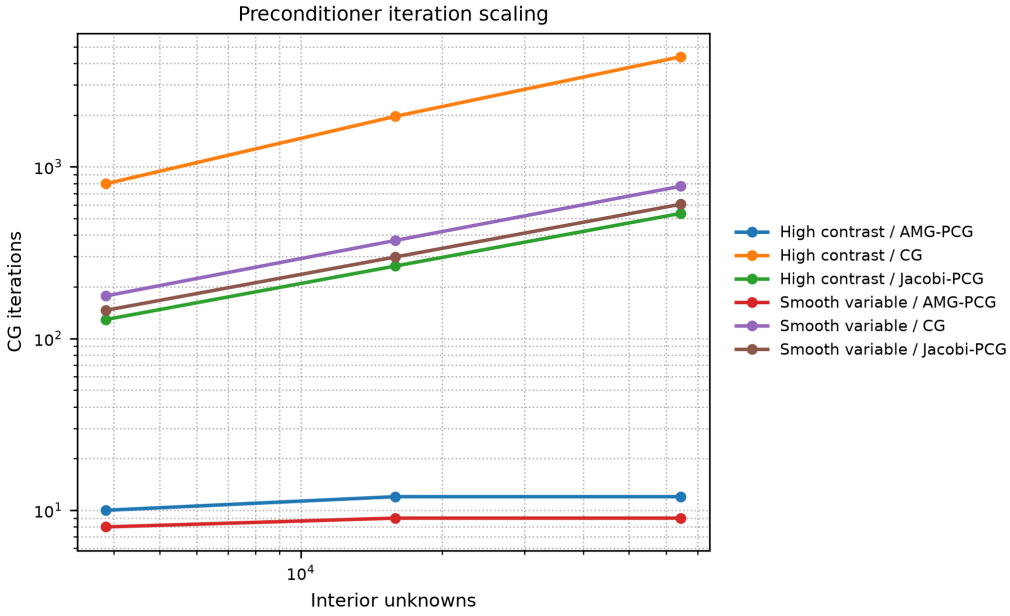

Figure 5: Iteration scaling for the same solver comparison. AMG-PCG keeps iteration counts nearly flat over the tested grid sizes for these cases.

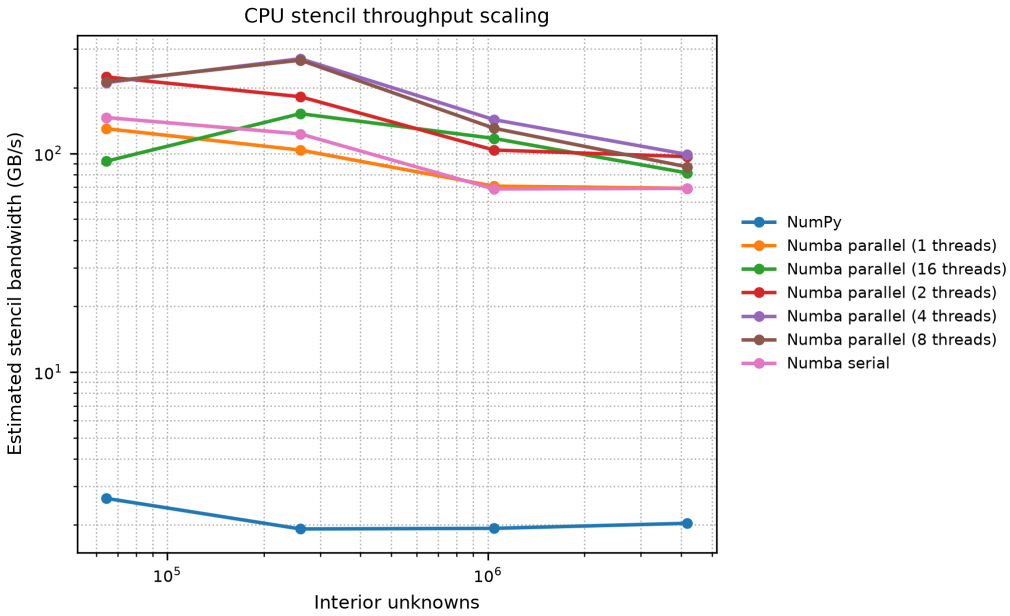

Figure 6: Estimated CPU stencil bandwidth for the variable-coefficient stencil apply.

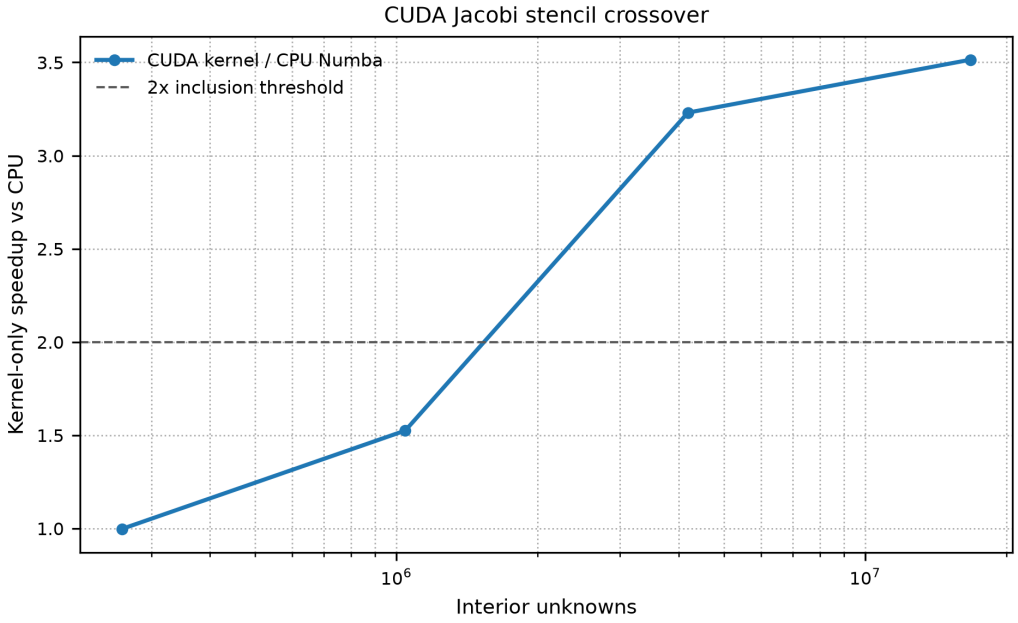

Figure 7: CUDA kernel-only speedup for repeated Jacobi updates relative to a four-thread Numba CPU baseline.